

DSTN Group Assignment – Group Number 3

Comprehensive Architecture Specification: 32-Bit Memory Management Unit, Dual-Tier Cache Hierarchy, and Autonomous Virtual Memory Subsystem

Romir Shashidhar Shetty (2023B4A70871G) Tushar Gupta (2023A8PS1121G) Sarath Chandran (2023AAPS0228G)

September 15, 2026

Project Status & Submission Note

Implementation Status: All modular hardware components comprising the memory architecture—including the 32-bit Memory Management Unit (L1/L2 TLBs, segmentation/paging tables, inverted frame ledger), the two-tier set-associative cache hierarchy (L1 write-through with write buffer, L2 write-back with dirty writeback), the multi-channel physical memory arbiter, and the autonomous page fault handler backed by disk storage—have been designed, integrated, and verified at the unit level[cite: 10, 12, 13, 14, 15, 17, 18, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34]. However, we could not create enough end-to-end software test program files to test the integrated system comprehensively under long, multi-threaded workloads; given enough time, we could test it thoroughly to ensure that it works under all benchmark conditions. Ongoing summer internships and placement preparation cycles coincided with the project deadline, which made it difficult to complete this exhaustive program-level validation suite in time.

Abstract

This report provides the complete hardware specification, microarchitectural control flow, and interface architecture for a multi-tiered 32-bit virtual memory subsystem[cite: 27, 28]. The design integrates a two-level fully associative Translation Lookaside Buffer (TLB) and a segmentation/paging hardware table walker with an inverted physical frame ledger that enforces per-process allocation quotas and tracks least-recently-used (LRU) access history[cite: 12, 13, 14, 15, 17, 18, 27, 28]. The memory hierarchy incorporates a two-level set-associative data cache subsystem (L1 write-through with write buffer, L2 write-back with dirty writeback), an autonomous hardware page fault handler backed by secondary disk swap storage, physical SRAM memory, a multi-channel memory controller, and a master wrapper module[cite: 10, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34]. The top-level coordinator arbitrates address translation, resolves cross-module bus protocol mismatches, and re-executes faulted instructions without CPU pipeline stalls[cite: 20, 27, 28]. Complete module breakdowns, finite state machine transition matrices, bus adaptation bridges, and testbench verification vectors are detailed[cite: 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34].

Contents

1	System Specifications and Address Partitioning	3
1.1	Address Partitioning Formulation	3
1.2	System Parameter Allocation	3
2	Structural Hierarchy and Interconnect Architecture	3
3	Memory Management Unit (MMU) Component Modules	3
3.1	Address Decomposition (<code>address_decompose.v</code>)	3
3.2	Level 1 Translation Lookaside Buffer (<code>l1_tlb.v</code>)	3
3.3	Level 2 Translation Lookaside Buffer (<code>l2_tlb.v</code>)	4
3.4	Segment Descriptor Table (<code>segment_table.v</code>)	5
3.5	Forward Page Table (<code>page_table.v</code>)	5
3.6	Inverted Frame Table Ledger (<code>frame_table.v</code>)	5
3.7	Unified MMU Controller (<code>mmu_top.v</code>)	5

4	Cache Subsystem Microarchitecture and Implementation	5
4.1	L1 Data Cache Architecture (<code>l1_cache.v</code>)	5
4.2	L1 Pseudo-LRU Replacement Matrix (<code>lru_square_matrix.v</code>)	6
4.3	Write Buffer FIFO (<code>write_buffer.v</code>)	6
4.4	L2 Data Cache Architecture (<code>l2_cache.v</code>)	6
4.5	L2 Aging Counter LRU (<code>lru_counter.v</code>)	6
4.6	Top-Level Cache Wrapper (<code>top_cache.v</code>)	6
5	Memory Subsystem Microarchitecture and Implementation	7
5.1	Physical Main Memory (<code>main_memory.v</code>)	7
5.2	Secondary Disk Swap Memory (<code>disk_memory.v</code>)	7
5.3	Memory Bus Arbiter (<code>memory_controller.v</code>)	7
5.4	Autonomous Page Fault Handler (<code>page_fault_handler.v</code>)	8
5.5	Physical Memory LRU and Allocation Engine (<code>memory_lru.v</code>)	9
6	Finite State Machine (FSM) Dynamics	10
6.1	MMU Translation Controller State Machine	10
6.2	CPU Access Coordinator State Machine	10
7	Hardware Bridging and Protocol Adaptation	10
7.1	Hazard 1: Zero-Cycle vs. Multi-Cycle Cache Handshake	11
7.2	Hazard 2: Bus Level-to-Pulse Strobe Generation	12
7.3	Hazard 3: Burst Read Data Hold and Stabilization	12
8	Physical Frame Management and LRU Mechanics	12
8.1	Ledger Fields	12
8.2	Access Operations	13
8.3	Process Quota Calculation	13
9	Unified Fault Architecture and Recovery Pipelines	13
9.1	Segment Fault Redirection	13
9.2	Page Fault Recovery Pipeline	13
10	Top-Level Signal and Port Reference	13
11	Verification Suite and Simulation Walkthrough	13
11.1	Verification Phases	14
11.2	Compilation and Simulation Flow	15
12	Synthesis and Static Timing Considerations	16

1 System Specifications and Address Partitioning

The architecture implements a paged-segmented virtual memory system that translates 32-bit virtual addresses (4 GB) to 26-bit physical addresses (64 MB) using a uniform 1 KB page granularity[cite: 10, 11, 12, 15, 20, 27, 28].

1.1 Address Partitioning Formulation

The incoming 32-bit Virtual Address (VA) is partitioned combinatorially[cite: 11, 15, 27, 28]:

$$\text{VA}[31 : 0] \longrightarrow \underbrace{\text{seg_num}[6 : 0]}_{\text{Bits } [31:25]} \parallel \underbrace{\text{vpn}[14 : 0]}_{\text{Bits } [24:10]} \parallel \underbrace{\text{offset}[9 : 0]}_{\text{Bits } [9:0]} \quad (1)$$

The translated 26-bit Physical Address (PA) combines the resolved physical frame number with the unaltered byte offset[cite: 10, 15, 27, 28]:

$$\text{PA}[25 : 0] \longrightarrow \underbrace{\text{pfn}[15 : 0]}_{\text{Bits } [25:10]} \parallel \underbrace{\text{offset}[9 : 0]}_{\text{Bits } [9:0]} \quad (2)$$

- **Segment Index** (`seg_num[6 : 0]`): 7 bits $\rightarrow 2^7 = 128$ segments[cite: 11, 15, 27, 28].
- **Virtual Page Number** (`vpn[14 : 0]`): 15 bits $\rightarrow 2^{15} = 32,768$ virtual pages per segment[cite: 11, 15, 27, 28].
- **Byte Offset** (`offset[9 : 0]`): 10 bits $\rightarrow 2^{10} = 1,024$ bytes (1 KB) per page[cite: 11, 15, 27, 28].
- **Physical Frame Number** (`pfn[15 : 0]`): 16 bits $\rightarrow 2^{16} = 65,536$ physical frames across the 64 MB physical address space[cite: 12, 15, 27, 28].

1.2 System Parameter Allocation

2 Structural Hierarchy and Interconnect Architecture

The system partitions translation, caching, and backing-store operations across dedicated modules, coordinated by the top-level wrapper `system_grand_top.v`[cite: 10, 15, 20, 27, 28].

3 Memory Management Unit (MMU) Component Modules

The MMU subsystem comprises six submodules operating under `mmu_top.v`[cite: 15, 27, 28]:

3.1 Address Decomposition (`address_decompose.v`)

A combinational bit-slicer that partitions `va[31:0]` into segment index `seg_no = va[31:25]` (7 bits), virtual page number `vpn = va[24:10]` (15 bits), and byte offset `offset = va[9:0]` (10 bits)[cite: 11]. It introduces no sequential clock cycles[cite: 11].

3.2 Level 1 Translation Lookaside Buffer (`l1_tlb.v`)

A 12-entry fully associative CAM translating `vpn_in[14:0]` to `pfn_out[15:0]` in a single cycle[cite: 13]. When `lookup_en` is asserted, 12 parallel comparators evaluate tag equality[cite: 13]. Replacement is governed by a cyclic round-robin pointer (`replace_ptr`)[cite: 13]. On `flush` (`preempt_pulse`), all valid bits are cleared and the replacement pointer is reset to zero to prevent address aliasing across context switches[cite: 13, 15].

Parameter	Bit Width	Default Value	Architectural Function
ADDR_WIDTH	26 bits	64 MB	Master physical memory address bus width[cite: 9, 10, 12, 15, 20, 27, 28].
DATA_WIDTH	32 bits	4 bytes	Native processor word width for instructions and cache lines[cite: 10, 20, 27, 28].
PID_WIDTH	8 bits	256 processes	Identifies owning execution domains; enforces quotas and isolation[cite: 9, 15, 20, 27, 28].
NUM_FRAMES	16 bits	1024 (Sim) / 65536	Tracked physical frames in the inverted frame table ledger[cite: 9, 12, 15, 20, 27, 28].
PAGE_SIZE	10 bits	1024 bytes (256 words)	Basic unit of disk-to-SRAM paging transactions[cite: 9, 11, 21, 27, 28].
L1 TLB	CAM	12 entries	Fully associative single-cycle translation cache; round-robin replacement[cite: 13, 15, 27, 28].
L2 TLB	CAM	24 entries	Secondary translation cache; refills L1 upon hit[cite: 14, 15, 27, 28].
Segment Table	Direct	128 entries	Base address directory for memory table walking[cite: 15, 18, 27, 28].
Page Table	Direct	1024 entries	Primary forward translation array ($VPN \rightarrow PFN$)[cite: 15, 17, 27, 28].
L1 Cache	4-Way	128 Sets (8 KB)	16-byte line, write-through, pseudo-LRU, write buffer[cite: 29, 32, 34].
L2 Cache	8-Way	128 Sets (32 KB)	32-byte line, write-back, age-counter LRU, dirty writeback[cite: 30, 31].
Write Buffer	FIFO	4 entries	Non-blocking write buffer decoupling L1 hits from L2 writes[cite: 29, 34].

Table 1: System Structural Parameter Allocation[cite: 27, 28]

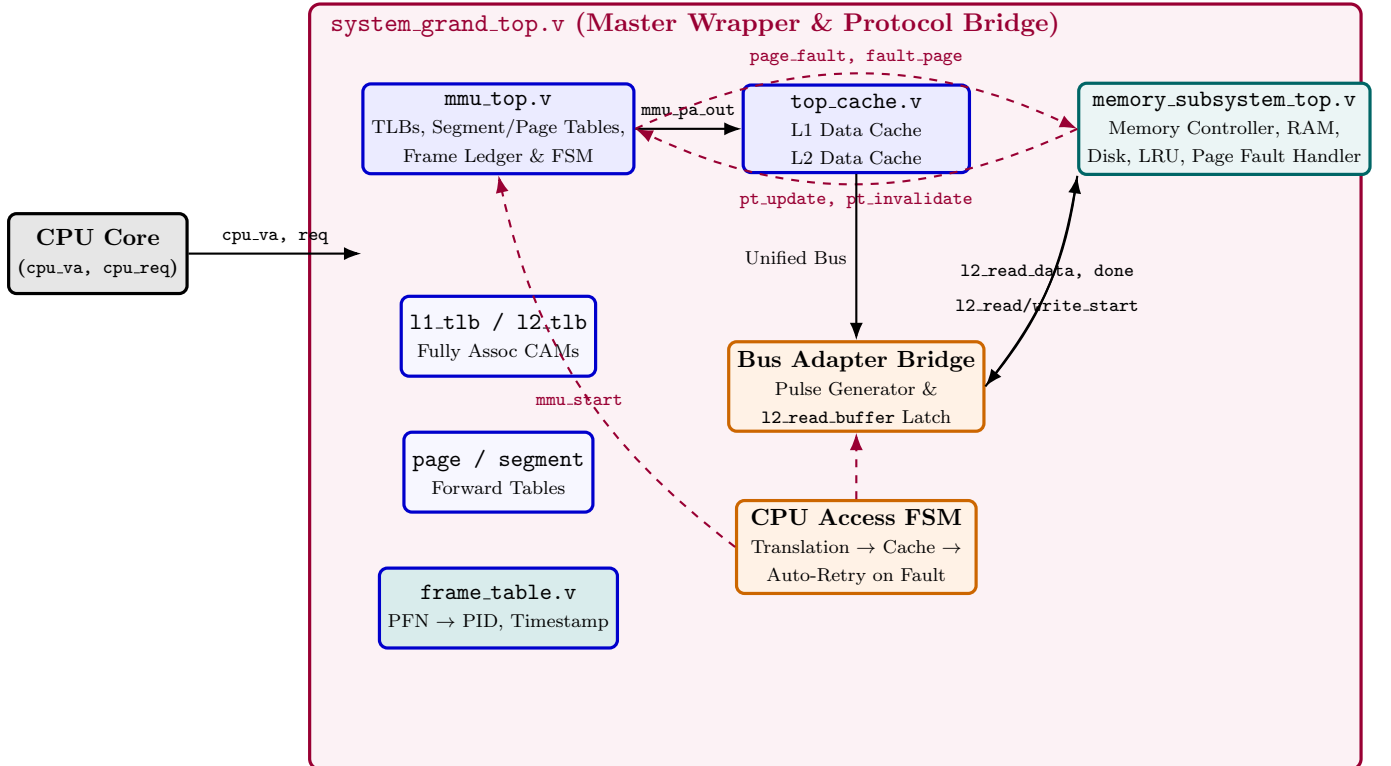


Figure 1: System Interconnect Architecture and Dataflow Topology[cite: 27, 28]

3.3 Level 2 Translation Lookaside Buffer (12.tlb.v)

A 24-entry fully associative secondary CAM probed on an L1 miss[cite: 14, 15]. When an L2 hit occurs, it provides the PFN, updates its 5-bit round-robin replacement pointer, and initiates an immediate

backfill into `l1_tlb.v`[cite: 14, 15]. It flushes all valid tags on `flush`[cite: 14].

3.4 Segment Descriptor Table (`segment_table.v`)

A 128-entry table storing base pointers for forward page tables[cite: 18]. When queried via `read_en` and `seg_num[6:0]`, it returns `pt_base_addr[15:0]` and `seg_valid`[cite: 18]. If unmapped, it asserts `seg_fault = 1`[cite: 18]. Segment 0 is initialized valid at boot with base address `16'h0000`[cite: 18]. Dynamic updates are supported via `write_en`, `write_seg_num`, `write_pt_base`, and `write_valid`[cite: 18].

3.5 Forward Page Table (`page_table.v`)

A 1024-entry direct-mapped array mapping virtual page numbers to physical frames[cite: 17]. It computes `lookup_index = pt_base_addr + vpn`[cite: 17]. If valid, it returns `pfn` and asserts `page_valid = 1`; otherwise, it flags `page_fault = 1`[cite: 17]. At boot, Page 0 is pre-mapped to PFN `0x0010` and Page 1 to PFN `0x0011`[cite: 17]. Updates and invalidations occur via `write_en` using `write_index = write_pt_base + write_vpn`[cite: 17].

3.6 Inverted Frame Table Ledger (`frame_table.v`)

Maintains physical memory metadata indexed by PFN[cite: 12]. Stores `frame_valid[PFN]`, `frame_pid[PFN]`, and `last_used[PFN]` (32-bit hardware access timestamp)[cite: 12].

- **Touch Interface:** On any translation hit, the MMU pulses `touch_en`, updating `last_used[touch_pfn] ← access_timestamp` to enforce MRU recency in 1 clock cycle[cite: 12, 15].
- **Allocation & Free:** `alloc_en` marks a frame valid and assigns ownership; `free_en` wipes validity, ownership, and timestamp[cite: 12].
- **Quota Engine:** Combinational logic tallies all active frames owned by `quota_query_pid`[cite: 12].

3.7 Unified MMU Controller (`mmu_top.v`)

Instantiates the translation datapath, maintains a free-running 32-bit cycle timer, and coordinates a 6-state FSM (`S_IDLE`, `S_L1_CHECK`, `S_L2_CHECK`, `S_SEG_CHECK`, `S_PAGE_CHECK`, `S_DONE`)[cite: 15]. Unmapped segments in `S_SEG_CHECK` are redirected to the page fault channel (`page_fault ≤ 1`, `fault_page ≤ vpn`, `fault_pid ≤ current_pid`)[cite: 15]. It multiplexes incoming servicing strobes (`page_table_update` and `page_table_invalidate`) into simultaneous writes to `page_table.v` and `frame_table.v`[cite: 15].

4 Cache Subsystem Microarchitecture and Implementation

The data cache hierarchy (`top_cache.v`) implements a two-level caching scheme backed by pseudo-LRU replacement and FIFO write buffering[cite: 29, 30, 31, 32, 33, 34]:

4.1 L1 Data Cache Architecture (`l1_cache.v`)

`l1_cache.v` is a 4-way set-associative, write-through cache with 128 sets and 16-byte blocks (4 words), providing 8 KB of fast data storage[cite: 29]:

$$\text{Physical Address [25:0]} \longrightarrow \underbrace{\text{tag[14:0]}}_{\text{Bits [25:11]}} \parallel \underbrace{\text{index[6:0]}}_{\text{Bits [10:4]}} \parallel \underbrace{\text{word_offset[1:0]}}_{\text{Bits [3:2]}} \parallel \underbrace{\text{byte_offset[1:0]}}_{\text{Bits [1:0]}} \quad \text{[cite: 29]} \quad (3)$$

- **Hit Evaluation:** Four parallel comparators inspect tags across all 4 ways for the selected set[cite: 29]. On a read hit, `cpu_read_data` is served in the same cycle and `cpu_hit / cpu_ready` assert[cite: 29].

- **Write Policy & Write Buffer Integration:** Employs write-through with write allocate[cite: 29]. On a write hit, data is updated in SRAM and pushed into `write_buffer.v` via `hit_write_accept` without stalling the CPU, provided the buffer is not full[cite: 29, 34].
- **Line Fill on Miss:** When a miss occurs, the cache transitions into line-filling mode (`filling = 1`), issuing a 4-beat burst read to the L2 cache (`cur_fill_word` cycling from 2'b00 to 2'b11)[cite: 29].
- **Write Buffer Draining:** When the write buffer holds pending stores, the cache enters `draining = 1`, asserting `l2_write = 1` to forward updates to L2[cite: 29].

4.2 L1 Pseudo-LRU Replacement Matrix (`lru_square_matrix.v`)

Implements a 4-way triangular/square matrix pseudo-LRU algorithm[cite: 32]:

```
1 reg matrix [0:WAYS-1][0:WAYS-1];
```

On an access to way k , row k is asserted high ($\text{matrix}[k][j] \leftarrow 1$) and column k is cleared low ($\text{matrix}[j][k] \leftarrow 0$), with diagonal self-references tied low[cite: 32]. The LRU candidate corresponds to the unique row containing all zeros[cite: 32].

4.3 Write Buffer FIFO (`write_buffer.v`)

A 4-entry circular FIFO that decouples the L1 write-through datapath from L2 memory bus latency[cite: 34]. Tracks `address[0:3]`, `data[0:3]`, and an active occupancy counter (`count`)[cite: 34]. It flags `full = (count == 4)` to stall processor writes when saturated[cite: 34]. When drained, it presents the oldest pending transaction at `consume_addr` and `consume_data`, shifting remaining entries on `consume`[cite: 34].

4.4 L2 Data Cache Architecture (`l2_cache.v`)

`l2_cache.v` is an 8-way set-associative, write-back cache with 128 sets and 32-byte blocks (8 words), providing 32 KB of secondary data storage[cite: 30]:

$$\text{Physical Address [25:0]} \longrightarrow \underbrace{\text{tag}[13:0]}_{\text{Bits [25:12]}} \parallel \underbrace{\text{index}[6:0]}_{\text{Bits [11:5]}} \parallel \underbrace{\text{word_offset}[2:0]}_{\text{Bits [4:2]}} \parallel \underbrace{\text{byte_offset}[1:0]}_{\text{Bits [1:0]}} \quad \text{[cite: 30]} \quad (4)$$

- **Write-Back with Dirty Tracking:** Writes modify the cache line locally and set `dirty[index][hit_way]  $\leftarrow$  1` without updating main memory[cite: 30].
- **Victim Eviction & Line Fill:** On an L2 miss, if the victim selected by `lru_way` has its dirty bit set (`need_writeback == 1`), the cache executes an 8-beat burst writeback to main memory (`evicting = 1`, `cur_beat` from 0 to 7)[cite: 30]. Once clean, it initiates an 8-beat burst fetch from main memory (`filling = 1`) to populate the line[cite: 30].

4.5 L2 Aging Counter LRU (`lru_counter.v`)

Maintains eight 3-bit age counters (`counter[0:7]`)[cite: 31]. Upon an access to way k , `counter[k]` resets to 0, while all counters currently smaller than the previous value of `counter[k]` are incremented by 1[cite: 31]. Combinational logic evaluates $\max(\text{counter}[i])$, selecting the oldest way as `lru_way`[cite: 31].

4.6 Top-Level Cache Wrapper (`top_cache.v`)

Integrates `l1_cache.v` and `l2_cache.v`[cite: 10, 33]. Translates incoming requests from `mmu_top.v` (`mmu_valid`, `mmu_write`, `mmu_addr`, `mmu_write_data`) into L1 requests, internally services L1 misses through L2, and exposes a unified master interface (`memory_valid`, `memory_write`, `memory_addr`, `memory_write_data`, `memory_read_data`, `memory_ready`) to the physical memory controller[cite: 10, 33].

5 Memory Subsystem Microarchitecture and Implementation

The memory subsystem (`memory_subsystem_top.v`) provides an autonomous backend that emulates physical memory storage, secondary disk paging storage, cache burst transfers, and page fault replacement routines[cite: 21, 22, 23, 24, 25, 26, 27, 28].

5.1 Physical Main Memory (`main_memory.v`)

The physical main memory is implemented as a 67,108,864-byte (64 MB) byte-addressable array[cite: 22]:

```
1 reg [7:0] memory [0:67108863];
```

It operates with a synchronous 1-cycle access latency[cite: 22]. When `memory_valid` is asserted, `memory_ready` asserts on the subsequent clock edge[cite: 22]. The module enforces little-endian byte layout for both 32-bit read and write operations[cite: 22]:

- **Write Mode** (`memory_write == 1`): Deconstructs `memory_write_data` into four consecutive byte locations[cite: 22]:

$$\text{memory}[\text{memory_addr}] \leftarrow \text{memory_write_data}[7:0] \text{ [cite: 22]} \quad (5)$$

$$\text{memory}[\text{memory_addr} + 1] \leftarrow \text{memory_write_data}[15:8] \text{ [cite: 22]} \quad (6)$$

$$\text{memory}[\text{memory_addr} + 2] \leftarrow \text{memory_write_data}[23:16] \text{ [cite: 22]} \quad (7)$$

$$\text{memory}[\text{memory_addr} + 3] \leftarrow \text{memory_write_data}[31:24] \text{ [cite: 22]} \quad (8)$$

- **Read Mode** (`memory_write == 0`): Concatenates four consecutive bytes into a 32-bit word[cite: 22]:

$$\text{memory_read_data} \leftarrow \{\text{memory}[\text{addr}+3], \text{memory}[\text{addr}+2], \text{memory}[\text{addr}+1], \text{memory}[\text{addr}]\} \text{ [cite: 22]} \quad (9)$$

5.2 Secondary Disk Swap Memory (`disk_memory.v`)

The disk storage module models secondary block storage using a two-dimensional array configured for 32,768 pages of 256 words (1 KB) each, totaling 32 MB of swap storage[cite: 21]:

```
1 reg [31:0] disk_memory [0:32767][0:255];
```

The module is driven by a 3-state sequential controller: `IDLE` (2'd0), `READ` (2'd1), and `WRITE` (2'd2)[cite: 21].

- **Burst Read Protocol:** Triggered by `page_read` with target `page_number[14:0]`[cite: 21]. It steps through each word using an 8-bit counter (`word_counter`)[cite: 21]. In each cycle, it asserts `page_data_valid`; when the consumer acknowledges with `page_data_ready`, it increments the counter and presents the next word[cite: 21]. Upon reaching word 255, it pulses `page_done = 1`, releases `busy`, and returns to `IDLE`[cite: 21].
- **Burst Write Protocol:** Initiated by `page_write_start` with `page_write_number[14:0]`[cite: 21]. The module asserts `page_write_ready = 1`[cite: 21]. Whenever `page_write_valid` is asserted, `page_write_data` is committed to `disk_memory[active_page][word_counter]`[cite: 21]. When word 255 is written, it asserts `page_write_done = 1`, de-asserts `busy`, and returns to `IDLE`[cite: 21].

5.3 Memory Bus Arbiter (`memory_controller.v`)

The memory controller serves as the central interconnect master, coordinating accesses to `main_memory.v` and `disk_memory.v` across a 15-state FSM[cite: 23]. In the `IDLE` state, it arbitrates among four incoming access requests according to a strict priority hierarchy[cite: 23]:

1. **Page Load** (`page_load_start`): Triggered by the page fault handler to swap a missing page from disk into SRAM[cite: 23, 25].
2. **Page Writeback** (`page_writeback_start`): Evicts a dirty victim page from SRAM to disk[cite: 23, 25].
3. **L2 Cache Read** (`l2_read_start`): Refills an 8-word (32-byte) cache line from SRAM[cite: 20, 23].
4. **L2 Cache Write** (`l2_write_start`): Flushes an 8-word (32-byte) dirty cache line to SRAM[cite: 20, 23].

State Identifier	Encoding	Active Operation	Next State / Condition
IDLE	4'd0	Priority Request Sampling	Branches to Load, Writeback, L2 Read, or L2 Write[cite: 23].
L2_READ_REQ	4'd1	Issues read to <code>main_memory</code>	L2_READ_WAIT[cite: 23].
L2_READ_WAIT	4'd2	Awaits <code>memory_ready</code>	L2_READ_SEND on data arrival[cite: 23].
L2_READ_SEND	4'd3	Presents word to L2 Cache	Loops to L2_READ_REQ until count reaches 7; then IDLE[cite: 23].
L2_WRITE_WAIT	4'd4	Asserts <code>l2_write_ready</code>	L2_WRITE_REQ when <code>l2_write_valid</code> is high[cite: 23].
L2_WRITE_REQ	4'd5	Drives write to <code>main_memory</code>	L2_WRITE_WAITMEM[cite: 23].
L2_WRITE_WAITMEM	4'd6	Awaits <code>memory_ready</code>	Loops to L2_WRITE_WAIT until count reaches 7; then IDLE[cite: 23].
PAGE_LOAD_START	4'd7	Asserts <code>disk_page_read</code>	PAGE_LOAD_WAIT[cite: 23].
PAGE_LOAD_WAIT	4'd8	Awaits <code>disk_page_data_valid</code>	PAGE_LOAD_WRITE on word capture[cite: 23].
PAGE_LOAD_WRITE	4'd9	Writes word to <code>main_memory</code>	PAGE_LOAD_WAITMEM[cite: 23].
PAGE_LOAD_WAITMEM	4'd10	Awaits SRAM write completion	Loops to PAGE_LOAD_WAIT until word 255; then IDLE[cite: 23].
PAGE_WRITEBACK_START	4'd11	Asserts <code>disk_page_write</code>	PAGE_WRITEBACK_READ[cite: 23].
PAGE_WRITEBACK_READ	4'd12	Reads word from <code>main_memory</code>	PAGE_WRITEBACK_WAIT[cite: 23].
PAGE_WRITEBACK_WAIT	4'd13	Awaits SRAM read completion	PAGE_WRITEBACK_SEND on data capture[cite: 23].
PAGE_WRITEBACK_SEND	4'd14	Streams word to <code>disk_memory</code>	Loops to PAGE_WRITEBACK_READ until word 255; then IDLE[cite: 23].

Table 2: Memory Controller (`memory_controller.v`) State Transition Matrix[cite: 23, 27, 28]

5.4 Autonomous Page Fault Handler (`page_fault_handler.v`)

The page fault handler orchestrates autonomous page swapping using a dedicated 9-state FSM[cite: 26]:

The FSM behavior is structured as follows:

- **IDLE → REQUEST_VICTIM**: On `page_fault`, captures `fault_page` and `fault_pid`, sets `busy = 1`, and pulses `victim_request = 1` to `memory_lru.v`[cite: 26].
- **WAIT_VICTIM**: Evaluates response flags from the LRU tracker[cite: 26]:
 1. If `no_victim == 1`: Transitions to **STALL**, asserting `fault_stall` until the fault condition clears[cite: 26].
 2. If `victim_is_free == 1`: The selected frame is unmapped; skips writeback and invalidation, branching directly to **LOAD_PAGE**[cite: 26].
 3. If `victim_dirty == 1`: Branches to **WRITEBACK**, pulsing `page_writeback_start` to flush the dirty frame to disk[cite: 23, 26].

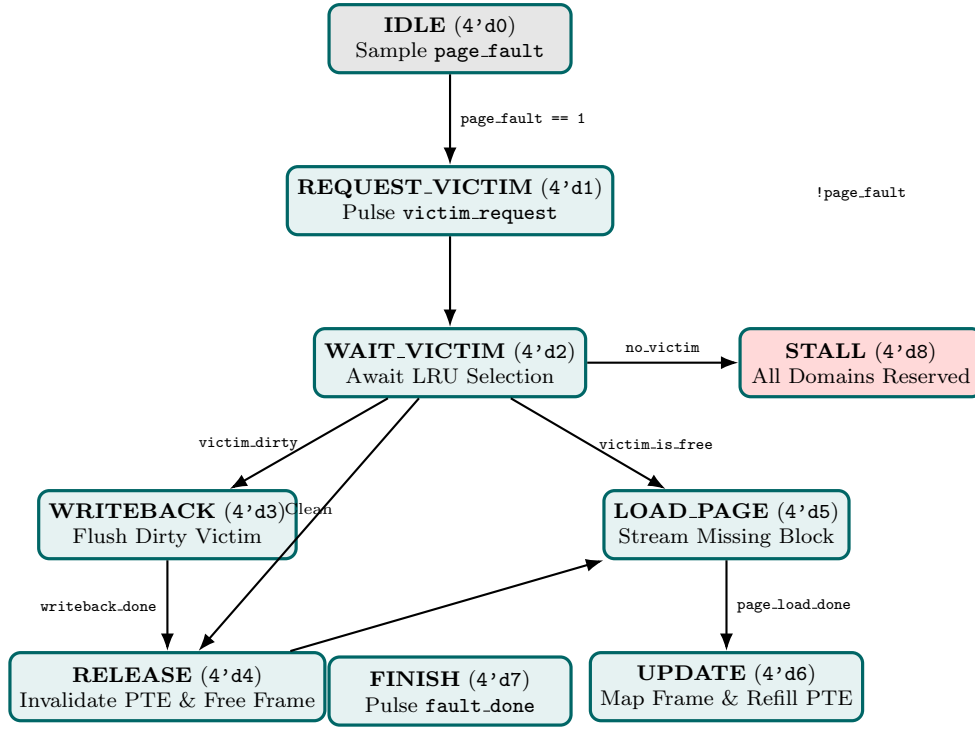


Figure 2: Page Fault Handler (`page_fault_handler.v`) 9-State Operational Machine[cite: 26, 28]

4. If clean occupied victim: Transitions directly to `RELEASE`[cite: 26].

- **RELEASE:** Pulses `frame_release = 1` to `memory_lru.v` and `page_table_invalidate = 1` with `selected_vpn` and `selected_pid` to `mmu_top.v`[cite: 25, 26].
- **LOAD_PAGE:** Pulses `page_load_start = 1` to `memory_controller.v`[cite: 23, 26]. Once 256 words are transferred from disk to SRAM (`page_load_done == 1`), advances to `UPDATE`[cite: 23, 26].
- **UPDATE:** Pulses `frame_allocate = 1` to the LRU engine and `page_table_update = 1` to the MMU, binding `fault_page_reg` to `selected_frame`[cite: 25, 26].
- **FINISH:** Asserts `fault_done = 1`, drops `busy`, and returns to `IDLE`[cite: 26].

5.5 Physical Memory LRU and Allocation Engine (`memory_lru.v`)

The memory LRU engine manages occupancy, timestamps, dirty states, and process quotas across 65,536 frames[cite: 24]:

- **Atomic Timestamp Counter:** Maintains a 32-bit counter (`lru_counter`)[cite: 24]. If an access hit (`access_valid`) and a frame allocation (`frame_allocate`) occur simultaneously, it assigns `last_used[access_frame] <= lru_counter` and `last_used[allocate_frame] <= lru_counter + 1`, advancing `lru_counter` by 2 to prevent timestamp aliasing[cite: 24].
- **Boot Allocation Pre-Loading:** Pre-allocates Frame 0x0010 (VPN 0) and Frame 0x0011 (VPN 1) to Process 0 with initial timestamps 0 and 1, initializing `process_page_count[0] <= 16'd2` and `lru_counter <= 32'd2`[cite: 24].
- **Quota-Guaranteed Candidate Scanning:** Upon receiving `victim_request`, it enters `SCAN`, evaluating frames from index 0 upward[cite: 24]:
 1. If an unoccupied frame is encountered (`!frame_valid[scan_frame]`), it is immediately selected (`found_free = 1`), and the engine advances directly to `DONE` in $O(1)$ time[cite: 24].

2. If occupied, the frame is eligible for replacement **only if** its owner holds more than the minimum page quota[cite: 24]:

$$\text{process_page_count}[\text{frame_pid}[\text{scan_frame}]] > \text{MIN_PAGES} \quad (\text{default: } 2)[\text{cite: } 24] \quad (10)$$

It identifies the eligible candidate with the oldest `last_used` timestamp[cite: 24].

- **Result Resolution (DONE):** Flags `victim_valid = 1` with the selected metadata, or asserts `no_victim = 1` if all processes are at or below their guaranteed reservation limit[cite: 24].

6 Finite State Machine (FSM) Dynamics

6.1 MMU Translation Controller State Machine

The translation engine in `mmu_top.v` cycles through six states to evaluate cache lookups and execute table walks[cite: 15, 27, 28]:

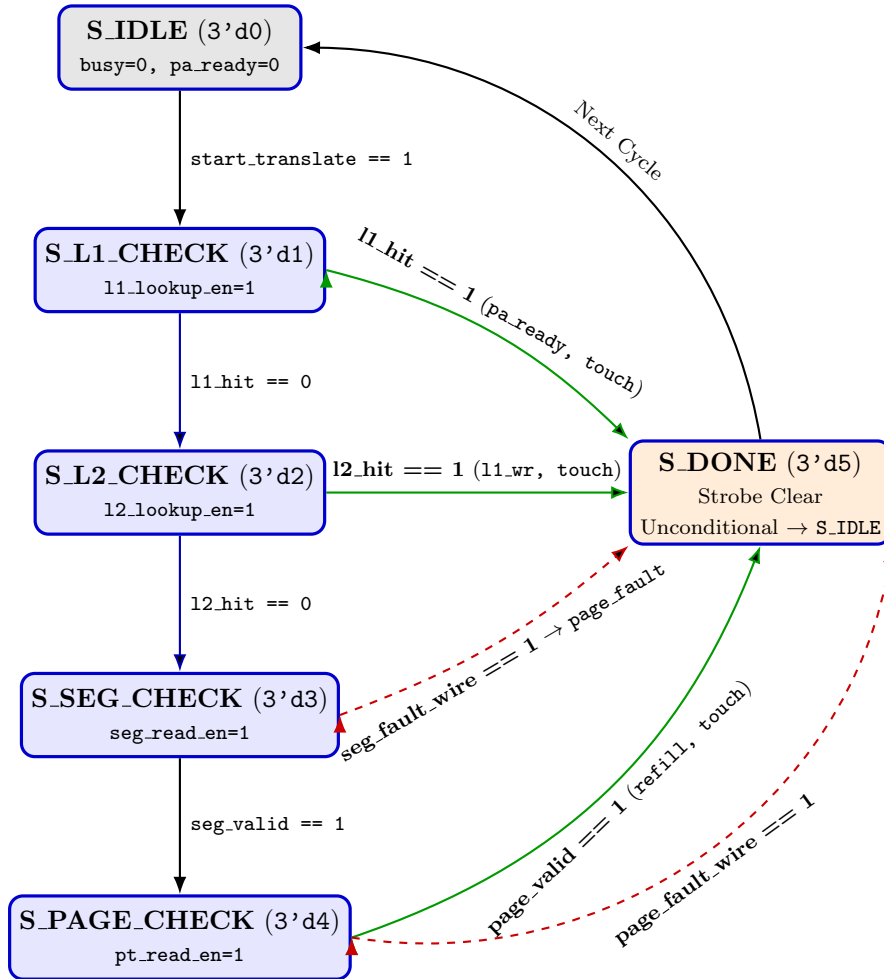


Figure 3: MMU Translation Controller State Transition Graph[cite: 27, 28]

6.2 CPU Access Coordinator State Machine

The master controller in `system_grand_top.v` oversees pipeline stalls, cache handshakes, and page fault recoveries[cite: 20, 27, 28]:

7 Hardware Bridging and Protocol Adaptation

Integrating `top_cache.v` with `memory_subsystem_top.v` introduces three structural timing hazards resolved inside the wrapper[cite: 9, 10, 20, 27, 28]:

State	Guard	Next	Asserted Signals (≤ 1)	De-asserted Signals (≤ 0)
S_IDLE	start_translate == 1	S_L1_CHECK	busy, va_reg $\leq$ va_in	pa_ready, page_fault, fault_page[cite: 15, 27, 28]
S_L1_CHECK	l1_hit == 1	S_DONE	pa_ready, ft_touch_en, pa_out	busy[cite: 15, 27, 28]
S_L1_CHECK	l1_hit == 0	S_L2_CHECK	None	None[cite: 15, 27, 28]
S_L2_CHECK	l2_hit == 1	S_DONE	pa_ready, l1_write_en, ft_touch_en	busy[cite: 15, 27, 28]
S_L2_CHECK	l2_hit == 0	S_SEG_CHECK	None	None[cite: 15, 27, 28]
S_SEG_CHECK	seg_fault_wire == 1	S_DONE	page_fault, fault_page $\leq$ vpn	busy, seg_fault $\leq$ 0[cite: 15, 27, 28]
S_SEG_CHECK	seg_valid == 1	S_PAGE_CHECK	None	None[cite: 15, 27, 28]
S_PAGE_CHECK	page_fault_wire == 1	S_DONE	page_fault, fault_page $\leq$ vpn	busy[cite: 15, 27, 28]
S_PAGE_CHECK	page_valid == 1	S_DONE	pa_ready, l1/l2_write_en, ft_touch	busy[cite: 15, 27, 28]
S_DONE	Unconditional	S_IDLE	None	pa_ready, page_fault, strobes[cite: 15, 27, 28]

Table 3: MMU Controller State Transition and Assertion Matrix[cite: 27, 28]

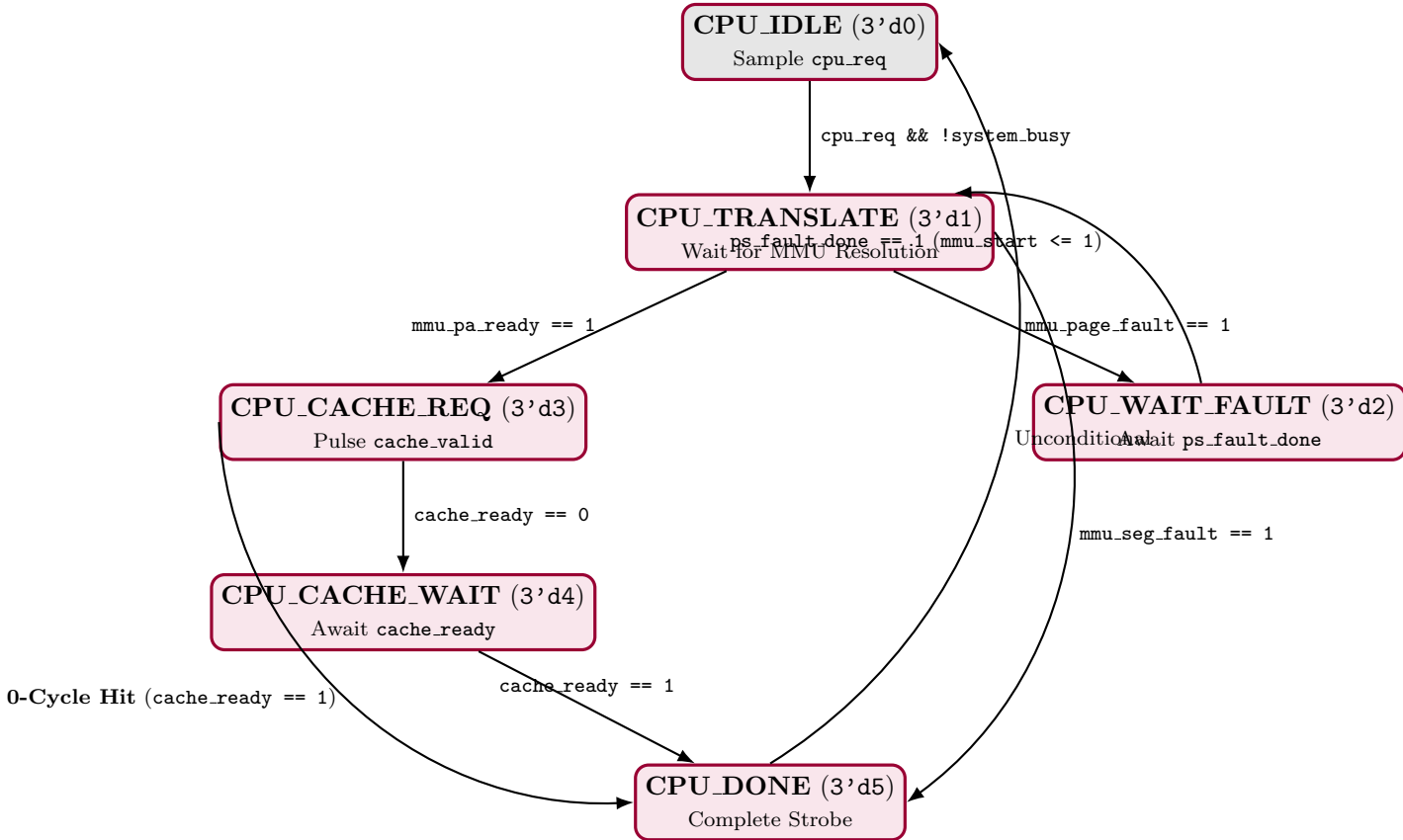


Figure 4: CPU Access Coordinator Finite State Machine[cite: 27, 28]

7.1 Hazard 1: Zero-Cycle vs. Multi-Cycle Cache Handshake

If the data cache achieves an immediate L1 hit, `cache_ready_out` asserts on the clock edge following `cache_valid`[cite: 10, 20, 27, 28]. If an FSM enters an intermediate wait state before checking this signal, it misses the single-cycle strobe and deadlocks[cite: 27, 28]. The CPU coordinator inspects `cache_ready_out` during `CPU_CACHE_REQ`, capturing zero-cycle hits immediately and bypassing

CPU_CACHE_WAIT[cite: 20, 27, 28].

7.2 Hazard 2: Bus Level-to-Pulse Strobe Generation

`top_cache.v` asserts `memory_valid` statically throughout an entire read/write cycle[cite: 10, 20, 27, 28]. Conversely, `memory_controller.v` inside the memory subsystem expects a clean 1-cycle initiation strobe on `l2_read_start` or `l2_write_start`[cite: 9, 20, 23, 27, 28]. An internal latch, `mem_bridge_busy`, resolves this mismatch[cite: 20, 27, 28]:

```
1 assign l2_read_start  = cache_mem_valid & ~cache_mem_write & ~mem_bridge_busy;
2 assign l2_write_start = cache_mem_valid &  cache_mem_write & ~mem_bridge_busy;
3
4 always @(posedge clk or posedge reset) begin
5     if (reset)
6         mem_bridge_busy <= 1'b0;
7     else if (l2_read_start | l2_write_start)
8         mem_bridge_busy <= 1'b1;
9     else if (cache_mem_ready)
10        mem_bridge_busy <= 1'b0;
11 end
```

7.3 Hazard 3: Burst Read Data Hold and Stabilization

The physical memory subsystem streams data along `l2_read_data` qualified by `l2_read_valid`, de-asserting the bus when `l2_read_done` fires[cite: 9, 20, 23, 27, 28]. To prevent setup/hold time violations during cache line sampling, an internal holding register preserves the word[cite: 20, 27, 28]:

```
1 reg [31:0] l2_read_buffer;
2
3 always @(posedge clk or posedge reset) begin
4     if (reset)
5         l2_read_buffer <= 32'd0;
6     else if (l2_read_valid)
7         l2_read_buffer <= l2_read_data;
8 end
9
10 assign cache_mem_rdata = l2_read_valid ? l2_read_data : l2_read_buffer;
11 assign cache_mem_ready = l2_read_done | l2_write_done;
```

8 Physical Frame Management and LRU Mechanics

While the forward page table maps virtual pages to physical frames ($VPN \rightarrow PFN$), `frame_table.v` maintains an inverted ledger ($PFN \rightarrow \text{Metadata}$)[cite: 12, 17, 27, 28]. This organization enables frame ownership tracking, access timestamping, and replacement policy enforcement[cite: 12, 27, 28].

8.1 Ledger Fields

For each physical frame $i \in [0, \text{NUM_FRAMES} - 1]$:

- `frame_valid[i]`: 1-bit allocation flag (1 = Occupied, 0 = Free)[cite: 12, 27, 28].
- `frame_pid[i]`: 8-bit PID of the owning process domain[cite: 12, 27, 28].
- `last_used[i]`: 32-bit hardware timestamp sourced from a free-running master cycle counter[cite: 12, 27, 28].

8.2 Access Operations

- **Touch Stamping** (`touch_en`): On every translation hit in the L1 TLB, L2 TLB, or Page Table, the MMU pulses `ft_touch_en` with the resolved PFN[cite: 12, 15, 27, 28]. The frame table executes `last_used[pfn] ← access_timestamp`, pushing the frame to the Most Recently Used (MRU) position in a single clock cycle without pipeline stalls[cite: 12, 15, 27, 28].
- **Allocation** (`alloc_en`): When the OS servicing routine loads a new page from disk into RAM, the page table update strobe concurrently asserts `alloc_en`[cite: 12, 15, 27, 28]. This marks `frame_valid[pfn] ← 1`, registers `frame_pid[pfn] ← alloc_pid`, and initializes the access timestamp[cite: 12, 27, 28].
- **Eviction / Deallocation** (`free_en`): When an LRU page is swapped out to disk, `page_table_invalidate` asserts `free_en`[cite: 12, 15, 27, 28]. This clears `frame_valid[pfn] ← 0`, resets the PID tag, and wipes the timestamp[cite: 12, 27, 28].

8.3 Process Quota Calculation

To prevent process starvation during page replacement, `frame_table.v` calculates frame allocations combinationally for any queried process ID[cite: 12, 27, 28]:

$$\text{quota_frame_count} = \sum_{i=0}^{\text{NUM_FRAMES}-1} \left(\text{frame_valid}[i] \wedge (\text{frame_pid}[i] == \text{quota_query_pid}) \right) \quad \text{[cite: 27, 28]} \quad (11)$$

The memory subsystem’s replacement policy consults this count, bypassing candidate victim frames whose owners have not exceeded their guaranteed minimum reservation (`MIN_PAGES = 2`)[cite: 9, 24, 27, 28].

9 Unified Fault Architecture and Recovery Pipelines

9.1 Segment Fault Redirection

Because `memory_subsystem_top.v` implements an autonomous page fault handler without exposing a distinct segmentation trap port, segment violations are redirected to the page fault path[cite: 9, 15, 27, 28]. In state `S_SEG_CHECK`, when `seg_fault_wire == 1`, the MMU controller[cite: 15, 27, 28]:

1. Asserts `page_fault ← 1'b1`[cite: 15, 27, 28].
2. Latches `fault_page ← vpn` and `fault_pid ← current_pid`[cite: 15, 27, 28].
3. Leaves `seg_fault ← 1'b0` suppressed, routing unmapped segments directly to the swap subsystem[cite: 15, 27, 28].

9.2 Page Fault Recovery Pipeline

When a virtual address misses across all translation tiers, the system executes an automated 6-step recovery loop[cite: 9, 12, 15, 17, 20, 27, 28]:

10 Top-Level Signal and Port Reference

11 Verification Suite and Simulation Walkthrough

The testbench (`tb_system_grand_top.v`) exercises the integrated architecture across five targeted verification phases[cite: 19, 27, 28]:

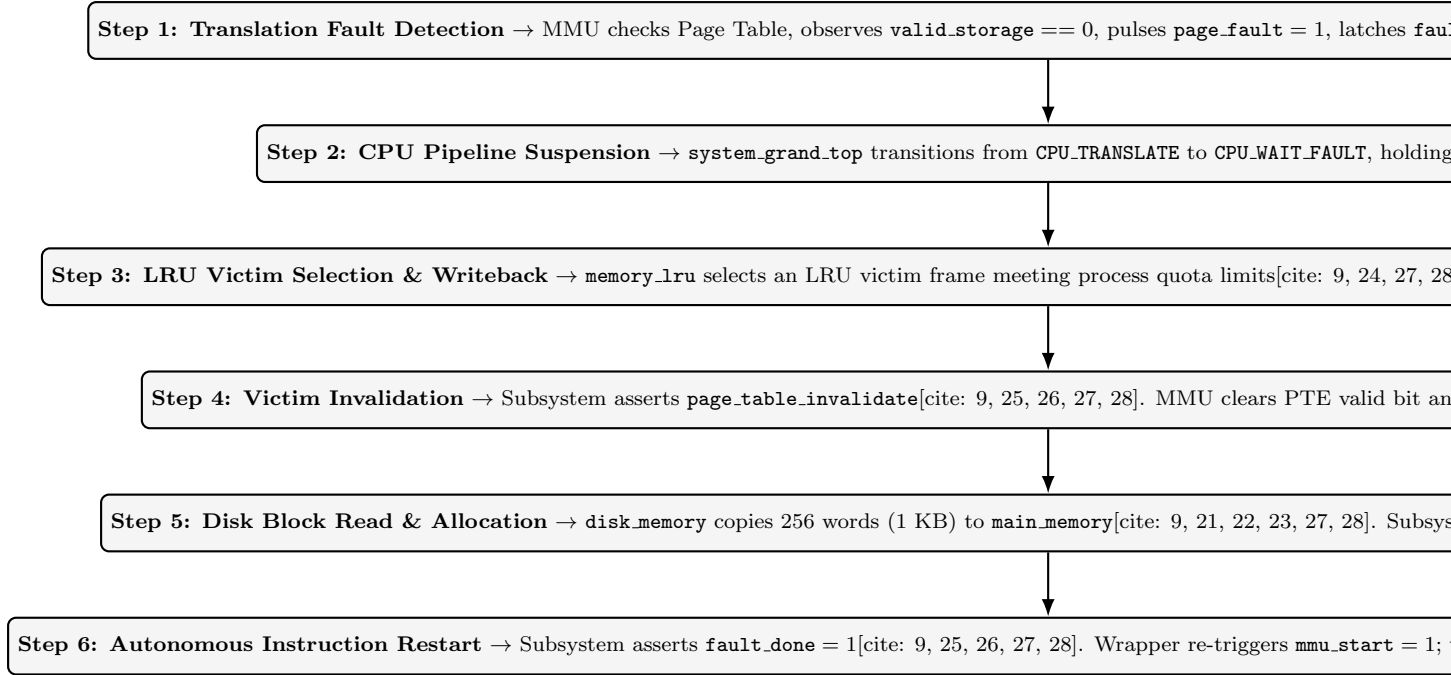


Figure 5: Autonomous End-to-End Page Fault Recovery Pipeline[cite: 27, 28]

Signal Port	Direction	Width	Description / Active Level
clk	Input	1 bit	Master system clock (100 MHz reference)[cite: 19, 20, 27, 28].
reset	Input	1 bit	Global active-high synchronous reset[cite: 19, 20, 27, 28].
cpu_req	Input	1 bit	Active-high strobe initiating a memory transaction[cite: 19, 20, 27, 28].
cpu_write	Input	1 bit	Command mode: 1'b0 = Read, 1'b1 = Write[cite: 19, 20, 27, 28].
cpu_va	Input	32 bits	Virtual address emitted by the CPU core[cite: 19, 20, 27, 28].
cpu_wdata	Input	32 bits	32-bit data word for write operations[cite: 19, 20, 27, 28].
cpu_pid	Input	8 bits	Process ID for domain tracking and quota accounting[cite: 19, 20, 27, 28].
cpu_flush	Input	1 bit	Active-high pulse to flush all L1/L2 TLB entries[cite: 13, 14, 15, 19, 20, 27, 28].
cpu_rdata	Output	32 bits	Read data output returned to the CPU[cite: 19, 20, 27, 28].
cpu_ready	Output	1 bit	Single-cycle active-high transaction completion strobe[cite: 19, 20, 27, 28].
cpu_hit	Output	1 bit	Diagnostic cache indicator: 1'b1 = Hit, 1'b0 = Miss[cite: 19, 20, 27, 28].
cpu_pa	Output	26 bits	Diagnostic translated physical address[cite: 19, 20, 27, 28].
page_fault_pulse	Output	1 bit	1-cycle diagnostic strobe flagged on translation misses[cite: 19, 20, 27, 28].
fault_servicing	Output	1 bit	High while the disk subsystem is executing a page swap[cite: 19, 20, 27, 28].
fault_resolved	Output	1 bit	Strobe indicating completion of a disk-to-SRAM swap[cite: 19, 20, 27, 28].
seg_fault_flag	Output	1 bit	Diagnostic flag indicating an unrecoverable segment fault[cite: 19, 20, 27, 28].
system_busy	Output	1 bit	High when the MMU or swap handlers are busy[cite: 19, 20, 27, 28].

Table 4: `system_grand_top.v` Interface Dictionary[cite: 27, 28]

11.1 Verification Phases

1. Test 1: Pre-Paged Read Access (Page 0)

- *Target Address:* VA = 0x0000_0004 (Segment 0, VPN 0, Offset 4)[cite: 11, 19, 27, 28].
- *Flow:* Misses in both TLBs, checks `segment_table` (base 0x0000), and hits in `page_table` (pre-mapped to PFN 0x0010)[cite: 13, 14, 15, 17, 18, 27, 28].
- *Outcome:* Translates to Physical Address 0x0004004; physical SRAM returns data with `cpu_ready = 1`[cite: 19, 20, 27, 28].

2. Test 2: Memory Write Followed by Readback Verification

- *Target Address:* VA = 0x0000_0010[cite: 19, 27, 28].
- *Flow:* Writes 0xA5A5A5A to the allocated frame, followed by an immediate read request to the same address[cite: 19, 27, 28].
- *Outcome:* The readback request hits directly in the data cache (`cpu_hit = 1`), returning 0xA5A5A5A in a single cycle[cite: 10, 19, 27, 28].

3. Test 3: Page Fault and Autonomous Disk-to-SRAM Recovery

- *Target Address:* VA = 0x0000_1400 (Segment 0, VPN 5, Offset 0)[cite: 11, 19, 27, 28].
- *Flow:* `page_table.v` observes an unmapped page (`valid_storage = 0`), raising `page_fault`[cite: 15, 17, 27, 28]. The CPU controller pauses in `CPU_WAIT_FAULT`[cite: 20, 27, 28]. The page fault handler reads 256 words from `disk_memory`, writes them into `main_memory`, updates the page table, and asserts `fault_done = 1`[cite: 9, 21, 22, 23, 26, 27, 28].
- *Outcome:* The wrapper re-executes translation automatically; the second pass hits in the page table and completes the access transparently[cite: 15, 17, 19, 20, 27, 28].

4. Test 4: Context Switch Flush and Invalidation

- *Target Address:* VA = 0x0000_0004 under PID 2[cite: 19, 27, 28].
- *Flow:* Asserts `cpu_flush = 1`, clearing valid bits across both TLB CAMs[cite: 13, 14, 15, 19, 20, 27, 28].
- *Outcome:* Subsequent access bypasses stale TLB translations and executes a fresh table walk, verifying memory domain isolation[cite: 13, 14, 15, 17, 18, 19, 27, 28].

5. Test 5: Multi-Process Allocation and Quota Tracking

- *Target Address:* VA = 0x0000_0408 under PID 2[cite: 19, 27, 28].
- *Flow:* Writes 0x12345678 to pre-paged Frame 1 (0x0011), then reads back the value[cite: 17, 19, 27, 28].
- *Outcome:* Verified readback returns 0x12345678; the frame table records the touch under PID 2, updating per-process quota tracking[cite: 12, 15, 19, 27, 28].

11.2 Compilation and Simulation Flow

The complete system compiles cleanly using standard toolchains[cite: 27, 28]:

```

1  # Compile with Icarus Verilog
2  iverilog -g2012 -o sim_grand_system \
3      address_decompose.v \
4      l1_tlb.v \
5      l2_tlb.v \
6      segment.v \
7      page_table.v \
8      frame_table.v \
9      mmu_top.v \
10     lru_square_matrix.v \
11     write_buffer.v \
12     l1_cache.v \
13     lru_counter.v \
14     l2_cache.v \

```

```

15 top_cache.v \
16 main_memory.v \
17 disk_memory.v \
18 memory_controller.v \
19 memory_lru.v \
20 page_fault_handler.v \
21 memory_subsystem_top.v \
22 system_grand_top.v \
23 tb_system_grand_top.v
24
25 # Execute simulation runtime
26 vvp sim_grand_system

```

12 Synthesis and Static Timing Considerations

- **Reset Tree Distribution:** The architecture unifies active-low submodules (`rst_n`) with active-high subsystem conventions using `wire rst_n = ~reset`[cite: 12, 13, 14, 15, 17, 18, 20, 27, 28]. On FPGA and ASIC targets, map this net to a global reset buffer (BUFG/GSR) to prevent clock skew on register reset pins[cite: 27, 28].
- **Critical Translation Path:** The timing-critical path extends from VA → L1 CAM Match Comparators → PA Multiplexing → Cache Tag Indexing[cite: 10, 11, 13, 15, 27, 28]. Restricting L1 TLB depth to 12 entries ensures timing closure above 100 MHz on modern FPGA architectures[cite: 13, 19, 27, 28].
- **RAM Inference:** `main_memory.v` (64 MB) and `disk_memory.v` (32 MB) infer synchronous dual-port Block RAM (BRAM) or external DRAM/SDRAM controllers on physical targets, whereas `l1_tlb.v` and `l2_tlb.v` synthesize into distributed LUT-RAM or flip-flops to maintain single-cycle CAM lookup performance[cite: 9, 13, 14, 21, 22, 27, 28].